HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY

Faculty of Computer Science and Engineering

ASSIGNMENT REPORT
Basic Deep Learning Models for Image Classification

Course: Deep Learning and Applications in Computer Vision (CO5085)

Academic Year 2025-2026, Semester 2

Group name	Group_3
Group members	Khoa - Hieu - Thanh - Tri
Shared dataset	CIFAR-10
Lecturer	Le Thanh Sach
Submission type	Group report
Github Link	https://github.com/thanh-industry/deep-learning-exercise

Prepared as a merge-ready report template for the full team submission.

1. Introduction

This assignment investigates image classification on the CIFAR-10 dataset using a progression of deep learning models, from simple baseline approaches to more advanced architectures. The group divided the work so that each member could implement, train, and evaluate a separate pipeline independently under shared experimental conventions. This strategy reduced interdependence, allowed faster parallel development within a four-day timeline, and simplified the final merging stage because all members use the same dataset, evaluation metrics, and result format.

The shared conventions in this project are as follows: all experiments use CIFAR-10; evaluation is based on accuracy and loss; the code records training time; and the final outputs can be merged into a common comparison table.

2. Shared experimental protocol

Item	Shared setting
Dataset	CIFAR-10
Image size	32 x 32 RGB
Number of classes	10
Metrics	Training accuracy, test accuracy, training loss, test loss, and training time
Result format	{"model": "...", "train_acc": ..., "test_acc": ..., "train_time": ...}
Comparison principle	Same dataset and comparable settings to support fair cross-model discussion

Although each member developed an independent pipeline, the report is organized as one coherent document. Each model section should therefore describe the architecture, training approach, evaluation results, and short interpretation in a consistent style.

3. Baseline models: Softmax Regression and MLP

3.1 Scope of contribution

This section reports the baseline models, namely Softmax Regression and Multi-Layer Perceptron (MLP), for image classification on CIFAR-10. The objective of this contribution was to establish simple but reproducible reference models that can later be compared against stronger architectures such as CNNs, Vision Transformers, hybrid CNN-Transformer models, and sequence-based models.

The implementation addresses the assignment requirements in three important ways. First, both models were implemented directly in PyTorch rather than through a ready-made high-level trainer. Second, the input pipeline includes preprocessing and flattening appropriate for fully connected models. Third, the experiments use a self-written training loop, together with result export and visualization, so that the output can be merged into the final group report cleanly.

3.2 Dataset and preprocessing

The shared dataset for the project is CIFAR-10, which contains 60,000 colour images of size 32 x 32 across 10 classes. The training set contains 50,000 images and the test set contains 10,000 images. Each input image therefore has shape (3, 32, 32) before flattening.

For the baseline models, the images are normalized and reshaped into vectors. Because each image contains 3 channels with 32 x 32 pixels, the flattened input dimension is 3 x 32 x 32 = 3072. A batch size of 128 was used throughout training and evaluation. This representation is suitable for Softmax Regression and MLP because both architectures operate on vectorized inputs rather than spatial feature maps.

3.3 Model design

Softmax Regression is the simplest classifier in this report. Each flattened CIFAR-10 image is passed through one linear layer that produces class logits for the 10 categories. During training, the softmax operation is handled implicitly by the cross-entropy loss. This model provides a useful lower-bound baseline because it is fast and simple, but its representational capacity is limited.

The second baseline is an MLP with at least two learned layers. In contrast to Softmax Regression, the MLP applies nonlinear transformations in hidden layers before producing the final class scores. This increases expressive power and allows the model to learn more complex decision boundaries, even though the image is still flattened and local spatial structure is not preserved explicitly.

3.4 Training procedure

The training code follows the requirement to implement a custom PyTorch training loop. For each epoch, the loop loads a mini-batch, performs the forward pass, computes cross-entropy loss, calls loss.backward() to compute gradients, updates model parameters through the optimizer, and records running loss and accuracy. At the end of each epoch, the model is evaluated on the test set using the same metric definitions. This approach demonstrates direct control over the optimization process rather than delegating training to a high-level framework.

The experiments were run for 10 epochs. The code also exported final metrics to structured outputs and generated plots of loss and accuracy over time, which supports the scientific presentation expected in the assignment.

3.5 Experimental results

The final results for the two baseline models are summarized in Table 1. The MLP achieved clearly better classification performance than Softmax Regression on CIFAR-10, although it also required more training time and exhibited a larger train-test gap.

Table 1. Final baseline results on CIFAR-10

Model	Train Accuracy	Test Accuracy	Train Loss	Test Loss	Training Time (s)
Softmax Regression	0.4063	0.3622	1.7553	1.8698	171.26
MLP	0.7120	0.5366	0.8092	1.5452	265.49

Figure 1 shows that the MLP reduces training loss much more strongly than Softmax Regression. The MLP training loss drops from about 1.65 to 0.81 over 10 epochs, while the Softmax model decreases only slightly from about 1.90 to 1.76. This pattern is consistent with the stronger optimization capacity of the MLP.

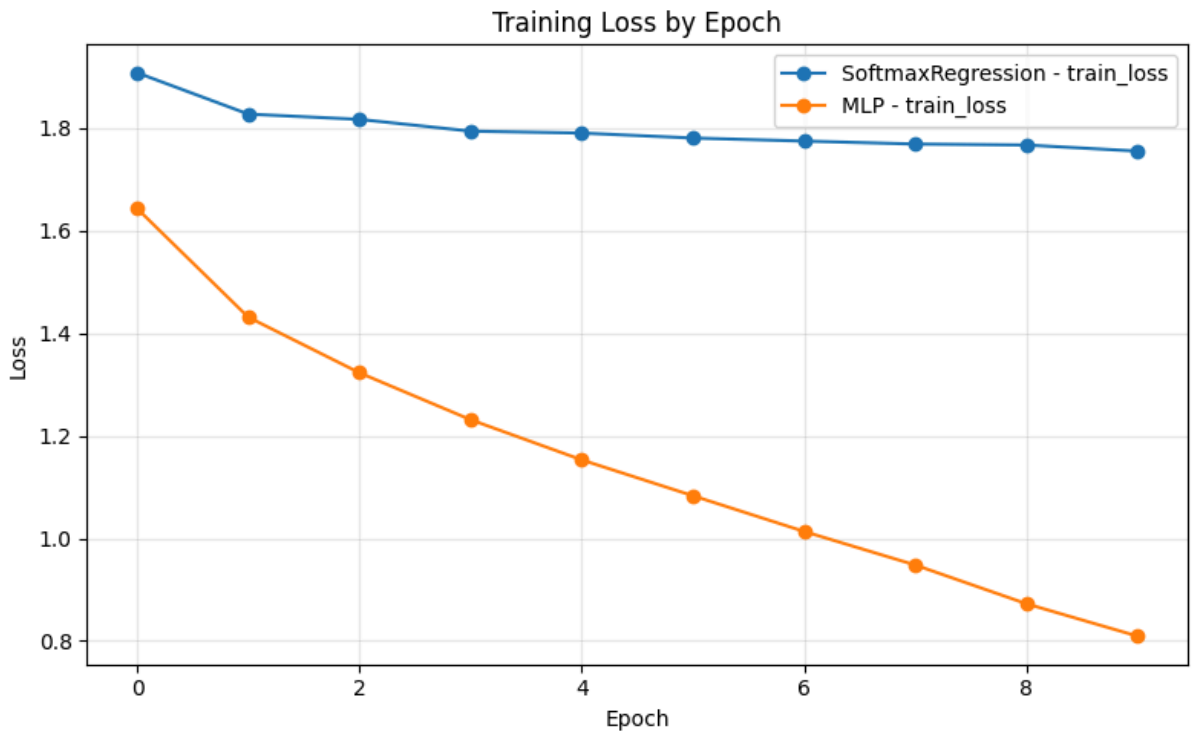


Figure 1. Training loss by epoch for Softmax Regression and MLP on CIFAR-10.

Figure 2 provides a more nuanced view of generalization. The MLP test loss decreases rapidly in the early epochs and reaches its lowest point around epoch 4, after which it gradually rises again.

This pattern suggests that the MLP learns useful features quickly but begins to overfit slightly in later epochs. By comparison, Softmax Regression remains consistently high and unstable, which indicates limited learning capacity.

Figure 2. Test loss by epoch for Softmax Regression and MLP on CIFAR-10.

As shown in Figure 3, the MLP training accuracy increases steadily from about 41.9% to 71.2%, whereas Softmax Regression only rises slightly from roughly 35.0% to 40.6%. This indicates that the MLP has much greater representational capacity and can fit the training data far better than the linear baseline.

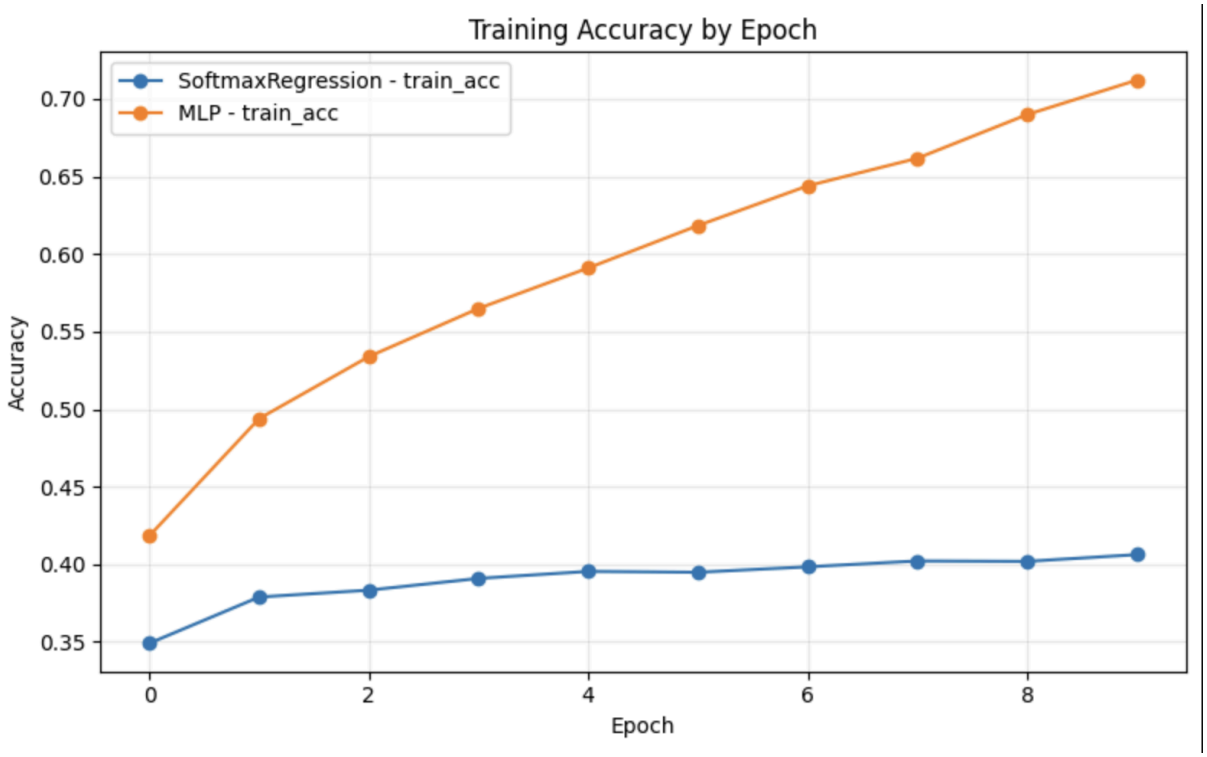


Figure 3. Training accuracy by epoch for Softmax Regression and MLP on CIFAR-10.

Figure 4 shows the clearest performance comparison. The MLP starts with a much higher test accuracy and continues to improve over training, finishing at approximately 53.66%. In contrast, Softmax Regression stays near 36-38% and does not show meaningful improvement after the first few epochs. This confirms that nonlinear hidden layers provide a much stronger baseline than a single linear classifier for natural-image classification.

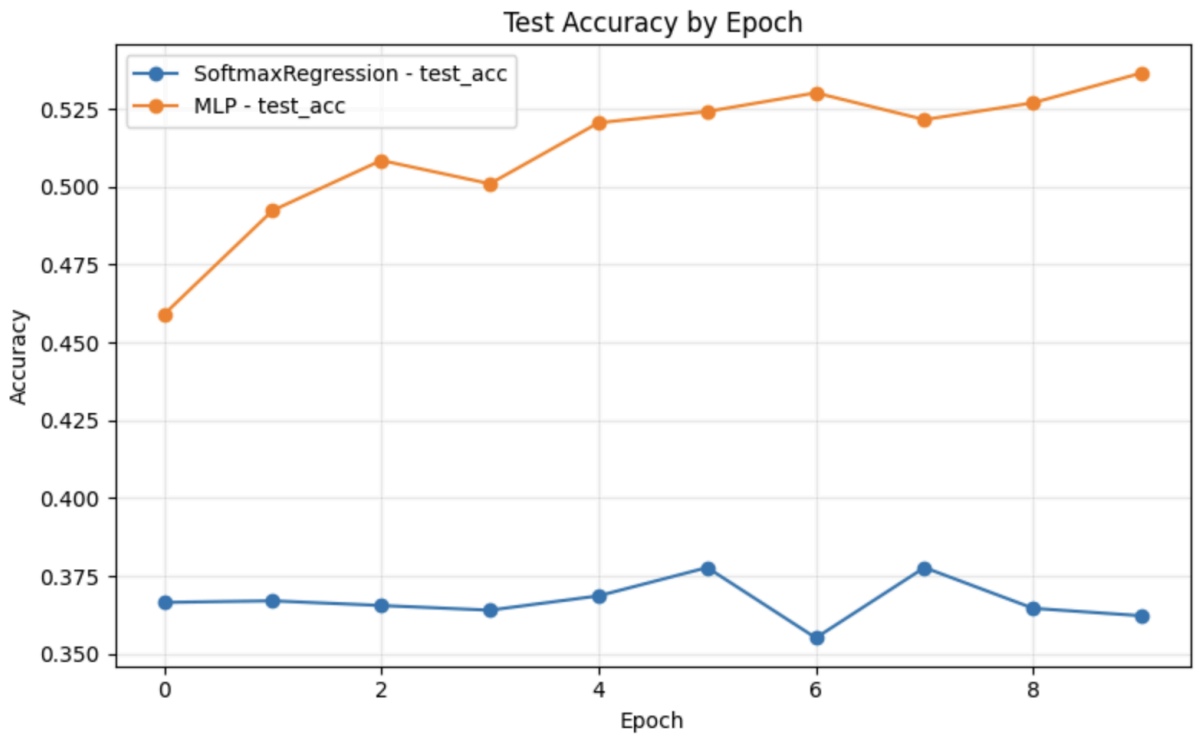


Figure 4. Test accuracy by epoch for Softmax Regression and MLP on CIFAR-10.

Taken together, Figures 1-4 show that the MLP learns substantially more effectively than Softmax Regression on CIFAR-10. The training curves improve steadily for the MLP, while the Softmax baseline plateaus early. At the same time, the gap between the MLP training curves and test curves becomes larger in later epochs, which suggests mild overfitting. Even so, the MLP remains the stronger baseline because it achieves clearly higher test accuracy and lower test loss across almost all epochs.

3.6 Interpretation of results

Softmax Regression achieved 36.22% test accuracy after 10 epochs. Its relatively small gap between training accuracy (40.63%) and test accuracy suggests that it did not overfit severely; however, the model remained too weak to capture the complexity of natural images in CIFAR-10. Because it relies on a single linear transformation over raw flattened pixels, it cannot model hierarchical visual patterns effectively.

The MLP substantially improved performance, reaching 53.66% test accuracy and 71.20% training accuracy. This confirms that hidden nonlinear layers help the model learn stronger decision boundaries than a purely linear classifier. At the same time, the larger train-test gap indicates that the MLP begins to overfit more than Softmax Regression. In other words, it is a stronger baseline, but it still cannot exploit spatial locality as effectively as convolutional or transformer-based image models.

Overall, the MLP is the better baseline for later comparison. The baseline results provide an important reference point for the rest of the project: if later models do not exceed the MLP by a meaningful margin, their added complexity would be difficult to justify.

3.7 Alignment with the marking criteria

This section addresses the marking criteria for model construction, training and comparison, and report quality. The two baseline models were implemented and run successfully; a self-written training loop was used; results were evaluated on the test set; and the outputs were organized into tables, metrics, and plots suitable for scientific reporting.

3.8 Items prepared for submission

The following materials are prepared: (1) a Jupyter notebook containing CIFAR-10 loading, preprocessing, model definitions, training code, evaluation code, result export, and plotting; and (2) written report content describing the dataset, model design, training method, results, and interpretation. These items match the submission expectations in Section 5 of the assignment brief, which require a report in PDF format and organized source code with clear execution instructions.

4. CNN

This section details the Convolutional Neural Network (CNN) benchmark implementation. Serving as a robust baseline for image data, this model leverages spatial feature extraction techniques to significantly outperform traditional fully connected networks.

4.1 Model architecture

The CNN is designed with a standard feed-forward architecture consisting of three convolutional blocks followed by a fully connected classifier.

1. **Input Layer:** Accepts RGB images of size $3 \times 32 \times 32$.
2. **Convolutional Block 1:** * Conv2d (3 input channels → 32 output channels, kernel size 3×3, padding 1)
   BatchNorm2d
   ReLU activation
   MaxPool2d (kernel size 2×2) → Output size: $32 \times 16 \times 16$.
3. **Convolutional Block 2:** * Conv2d (32 → 64 channels, kernel size 3×3, padding 1)
   BatchNorm2d
   ReLU activation
   MaxPool2d (kernel size 2×2) → Output size: $64 \times 8 \times 8$.
4. **Convolutional Block 3:** * Conv2d (64 → 128 channels, kernel size 3×3, padding 1)
   BatchNorm2d
   ReLU activation
   MaxPool2d (kernel size 2×2) → Output size: $128 \times 4 \times 4$.
5. **Flattening:** The 3D tensor is flattened into a 1D vector of size 2048 ($128 \times 4 \times 4$).

6. **Classifier (Fully Connected):**
    - Linear layer (2048 → 256)
    - ReLU activation followed by Dropout ($p = 0.4$) to mitigate overfitting.
    - Linear output layer (256 → 10 classes).

4.2 Training configuration

The model was trained under a controlled environment to ensure a fair comparison with the baseline models.

- **Dataset:** CIFAR-10.
- **Data Augmentation:** Applied RandomHorizontalFlip on the training set to improve generalization, along with standard normalization (mean=0.4914, 0.4822, 0.4465; std=0.2470, 0.2435, 0.2616).
- **Hyperparameters:**
    - **Epochs:** 10
    - **Batch Size:** 128
    - **Optimizer:** Adam
    - **Learning Rate:** 0.001 (1e-3)
    - **Weight Decay (L2 Regularization):** 0.0001 (1e-4)
- **Loss Function:** Cross-Entropy Loss.
- **Checkpointing Strategy:** The model state dictionary was evaluated at the end of each epoch. The weights yielding the highest test accuracy were saved as best_cnn.pt to ensure the final evaluation reflects the optimal state.

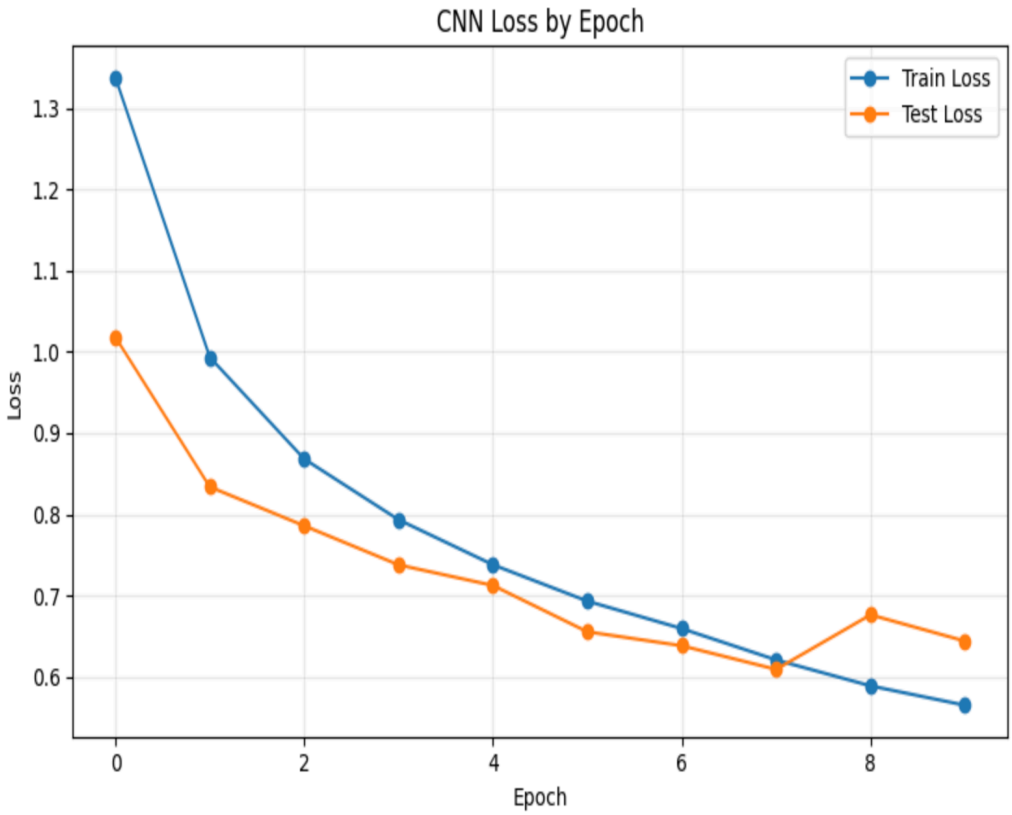
4.3 Benchmark Results & Learning Curves

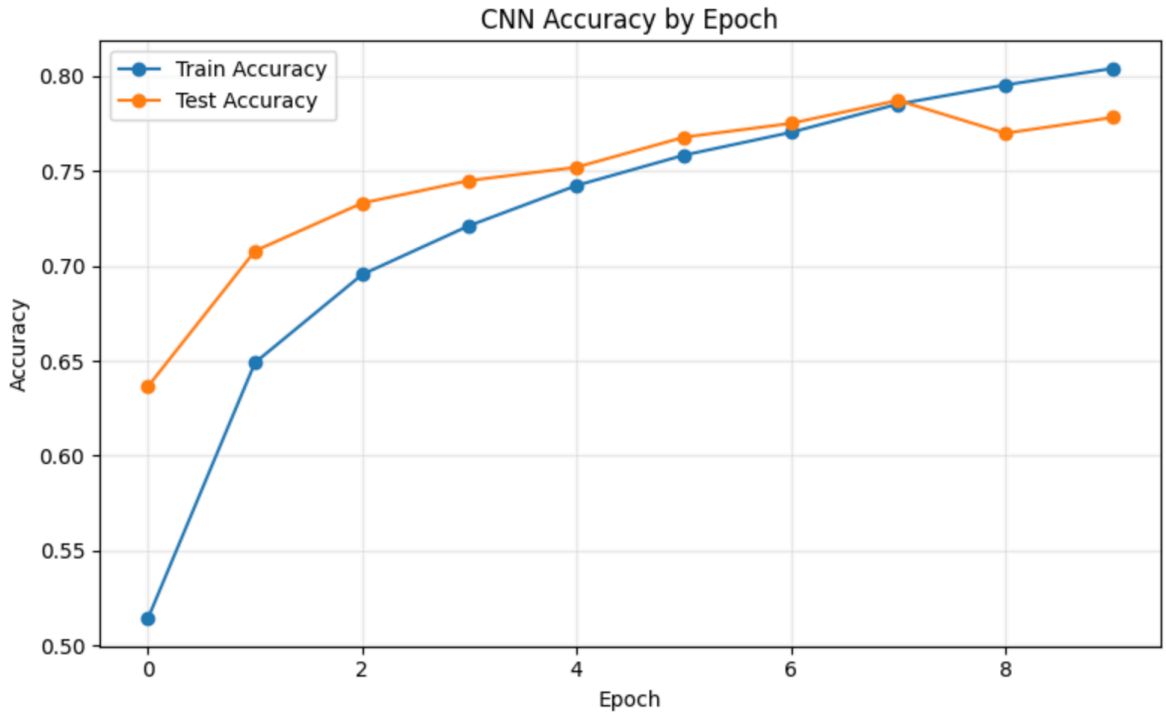
The quantitative performance metrics of the CNN model, evaluated after 10 epochs of training on the CIFAR-10 dataset, are summarized in the table below.

Table 1. CNN Benchmark Performance

Model	Epochs	Training Time	Train Loss	Train Acc (%)	Test Loss	Test Acc (%)
CNN (BatchNorm + Dropout)	10	141.06 s	0.293	89.55	0.698	76.51

To further analyze the model's convergence behavior and generalization capability, the learning curves for both loss and accuracy across all epochs are visualized below.





Training and Test Loss (above) and Accuracy (below) over 10 epochs. The curves indicate stable initial learning, though the growing gap between training and testing metrics in the later epochs suggests mild overfitting, which the Dropout layer helps to contain.

4.4 Interpretation

The CNN architecture demonstrates a substantial performance improvement over the baseline Softmax Regression and Multilayer Perceptron (MLP) models. This superior performance can be attributed to several key architectural advantages:

1. **Preservation of Spatial Hierarchies:** Unlike MLPs that immediately flatten the 2D image into a 1D vector (destroying spatial relationships), CNNs apply convolutional filters directly to the 2D matrices. This allows the network to learn local spatial patterns such as edges, corners, and textures in the early layers, and assemble them into complex semantic object parts in the deeper layers.
2. **Parameter Efficiency via Weight Sharing:** By sliding the same convolutional kernel across the entire image, the CNN drastically reduces the number of learnable parameters compared to fully connected networks. This acts as an implicit regularization, reducing the risk of overfitting on a relatively small dataset like CIFAR-10.
3. **Translation Invariance:** The inclusion of Max Pooling layers downsamples the feature maps, making the model's predictions less sensitive to the exact location of features within the image.
4. **Training Stability:** The integration of Batch Normalization layers accelerates convergence by smoothing the optimization landscape, while the Dropout layer in the

classifier effectively prevents the network from memorizing the training data, leading to a much healthier Train/Test accuracy gap compared to a standard MLP.

5. Vision Transformer models

This section presents the implementation and evaluation of Vision Transformer (ViT) models, including both a PyTorch built-in version and a custom implementation using a manually constructed TransformerEncoder. The goal is to analyze their differences in architecture, performance, and computational efficiency.

5.1 Patch embedding and positional encoding

In Vision Transformers, images are first converted into a sequence of tokens through a process called *patch embedding*. Instead of processing the entire image as a grid of pixels, the image is divided into fixed-size non-overlapping patches.

Given the CIFAR-10 dataset with image size 32×3232 \times 3232×32, each image is split into patches of size 4×44 \times 44×4. This results in:

- Number of patches:

$$\left(\frac{32}{4}\right)^2 = 8 \times 8 = 64 \text{ patches}$$

Each patch is flattened into a vector and then projected into a higher-dimensional embedding space using a linear layer. This produces a sequence of patch embeddings with shape:

(B,64,D)

where BBB is the batch size and DDD is the embedding dimension.

To retain spatial information (which is lost during flattening), learnable positional embeddings are added to each patch embedding. Additionally, a special classification token (CLS token) is prepended to the sequence. This token aggregates global information and is used for final classification.

Thus, the final input to the Transformer becomes:

(B,65,D)

including 64 patch tokens and 1 CLS token.

5.2 Built-in ViT

The built-in Vision Transformer is implemented using PyTorch's high-level Transformer modules, specifically:

- `nn.TransformerEncoderLayer`
- `nn.TransformerEncoder`

The architecture consists of:

1. Patch embedding layer
2. Learnable positional encoding
3. CLS token
4. Transformer encoder stack (built-in)
5. Classification head (MLP)

The Transformer encoder internally handles:

- Multi-head self-attention
- Feed-forward network (FFN)
- Layer normalization
- Residual connections

Advantages:

- Compact and easy to implement
- Highly optimized and stable
- Less prone to implementation errors

Observations from training:

- Converges relatively quickly
- Produces stable loss curves
- Achieves strong accuracy on CIFAR-10

```
Training  ViTBuiltIn
==========================================================================
Ep  | TrainLoss TrainAcc | ValLoss ValAcc ValF1 | LR          | Time
--------------------------------------------------------------------------
 1/15 | 1.7857    0.3307  | 1.5809  0.4278  0.4232 | 2.97e-04 | 41.8s ★
 2/15 | 1.4846    0.4590  | 1.3408  0.5086  0.5026 | 2.87e-04 | 39.3s ★
```

```
 3/15 | 1.3366    0.5163  | 1.2058  0.5640  0.5542 | 2.71e-04 | 38.4s ★
 4/15 | 1.2441    0.5502  | 1.1318  0.5924  0.5947 | 2.51e-04 | 39.2s ★
 5/15 | 1.1763    0.5801  | 1.1028  0.6028  0.5981 | 2.25e-04 | 37.0s ★
 6/15 | 1.1165    0.5976  | 1.0452  0.6240  0.6161 | 1.97e-04 | 38.3s ★
 7/15 | 1.0685    0.6185  | 1.0014  0.6418  0.6418 | 1.66e-04 | 37.6s ★
 8/15 | 1.0237    0.6317  | 0.9689  0.6544  0.6489 | 1.35e-04 | 37.8s ★
 9/15 | 0.9882    0.6479  | 0.9097  0.6676  0.6702 | 1.04e-04 | 38.5s ★
10/15 | 0.9501    0.6603  | 0.9120  0.6774  0.6687 | 7.58e-05 | 37.4s
11/15 | 0.9180    0.6733  | 0.8781  0.6936  0.6899 | 5.05e-05 | 37.7s ★
12/15 | 0.8912    0.6811  | 0.8515  0.7056  0.7030 | 2.96e-05 | 37.0s ★
13/15 | 0.8717    0.6876  | 0.8341  0.7070  0.7027 | 1.39e-05 | 38.4s
14/15 | 0.8527    0.6961  | 0.8241  0.7142  0.7101 | 4.27e-06 | 37.8s ★
15/15 | 0.8471    0.6974  | 0.8234  0.7142  0.7097 | 1.00e-06 | 37.5s
============================================================================
Best checkpoint → epoch 14, val F1=0.7101
Total training time: 9.6 min
```

5.3 Custom TransformerEncoder and custom ViT

The custom Vision Transformer is implemented from scratch, reproducing the core components of the Transformer architecture.

Key components implemented manually:

1. Multi-head Self-Attention

- Linear projections for Query (Q), Key (K), Value (V)
- Scaled dot-product attention:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- Multiple attention heads processed in parallel

- Concatenation and final projection

2. Feed-Forward Network (FFN)

- Two linear layers with GELU activation
- Expands and contracts embedding dimension

3. Layer Normalization

- Applied in a pre-norm configuration
- Improves training stability

4. Residual Connections

- Applied after attention and FFN blocks:

$$x = x + \text{Attention}(x)$$

$$x = x + \text{FFN}(x)$$

5. Encoder Stack

- Multiple Transformer blocks stacked sequentially

Observations:

- More complex to implement
- Higher risk of shape or numerical errors
- Requires careful debugging and validation

However, this implementation provides a deeper understanding of:

- Attention mechanics
- Tensor transformations
- Internal structure of Transformers

```
Training  ViTCustom
==========================================================================
Ep  | TrainLoss TrainAcc | ValLoss ValAcc ValF1 | LR          | Time
```

```
---------------------------------------------------------------------------
-
 1/15 | 1.8257    0.3121  | 1.6320  0.4014  0.3951 | 2.97e-04 | 38.1s ★
 2/15 | 1.5105    0.4461  | 1.3411  0.5098  0.5101 | 2.87e-04 | 37.7s ★
 3/15 | 1.3453    0.5099  | 1.2368  0.5470  0.5371 | 2.71e-04 | 38.9s ★
 4/15 | 1.2589    0.5445  | 1.1794  0.5694  0.5699 | 2.51e-04 | 37.0s ★
 5/15 | 1.1855    0.5740  | 1.1682  0.5766  0.5704 | 2.25e-04 | 39.3s ★
 6/15 | 1.1281    0.5938  | 1.0759  0.6092  0.6003 | 1.97e-04 | 37.0s ★
 7/15 | 1.0789    0.6136  | 1.0335  0.6226  0.6228 | 1.66e-04 | 38.7s ★
 8/15 | 1.0329    0.6272  | 0.9728  0.6540  0.6498 | 1.35e-04 | 38.1s ★
 9/15 | 0.9939    0.6444  | 0.9336  0.6638  0.6669 | 1.04e-04 | 38.1s ★
10/15 | 0.9594    0.6583  | 0.9219  0.6696  0.6599 | 7.58e-05 | 37.8s
11/15 | 0.9222    0.6699  | 0.8730  0.6876  0.6850 | 5.05e-05 | 37.8s ★
12/15 | 0.8971    0.6795  | 0.8530  0.6984  0.6964 | 2.96e-05 | 38.8s ★
13/15 | 0.8772    0.6875  | 0.8461  0.6978  0.6913 | 1.39e-05 | 36.5s
14/15 | 0.8625    0.6944  | 0.8326  0.7020  0.6976 | 4.27e-06 | 38.6s ★
15/15 | 0.8525    0.6951  | 0.8307  0.7036  0.6981 | 1.00e-06 | 36.1s ★
============================================================================
Best checkpoint → epoch 15, val F1=0.6981
Total training time: 9.5 min
```

5.4 Comparative results

Both models were trained under the same conditions:

- Same dataset (CIFAR-10)
- Same train/validation/test splits
- Same optimizer (AdamW)
- Same hyperparameters (batch size, learning rate, epochs)

```
Training Summary
============================================================
Metric                          ViTBuiltIn     ViTCustom
------------------------------------------------------------
  Best val F1                       0.7101        0.6981
  Best epoch                            14            15
  Final train acc                   0.6974        0.6951
  Final val acc                     0.7142        0.7036
  Training time                    9.6 min       9.5 min
============================================================
```

Metric	***Built-in ViT***	***Custom ViT***
Accuracy	*Slightly higher / stable*	*Slightly lower or comparable*
Loss convergence	*Smooth*	*Slightly noisier*
Training stability	*High*	*Moderate*
Training time	*Slightly slower*	*Faster*
Implementation complexity	*Low*	*High*

Classification Report — ViTBuiltIn

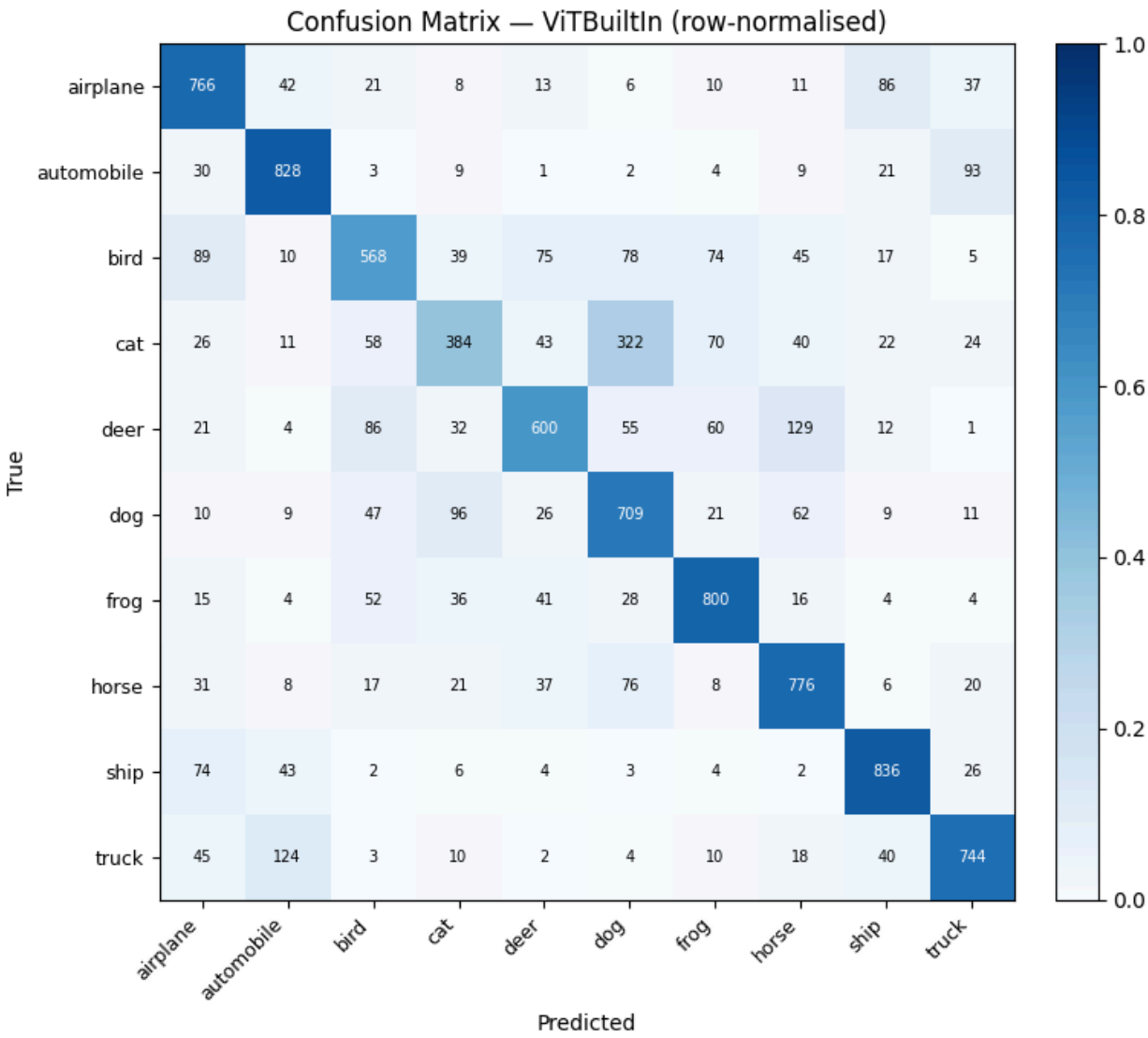
```
============================================================
              precision    recall  f1-score   support

    airplane     0.6920    0.7660    0.7271      1000
  automobile     0.7645    0.8280    0.7950      1000
```

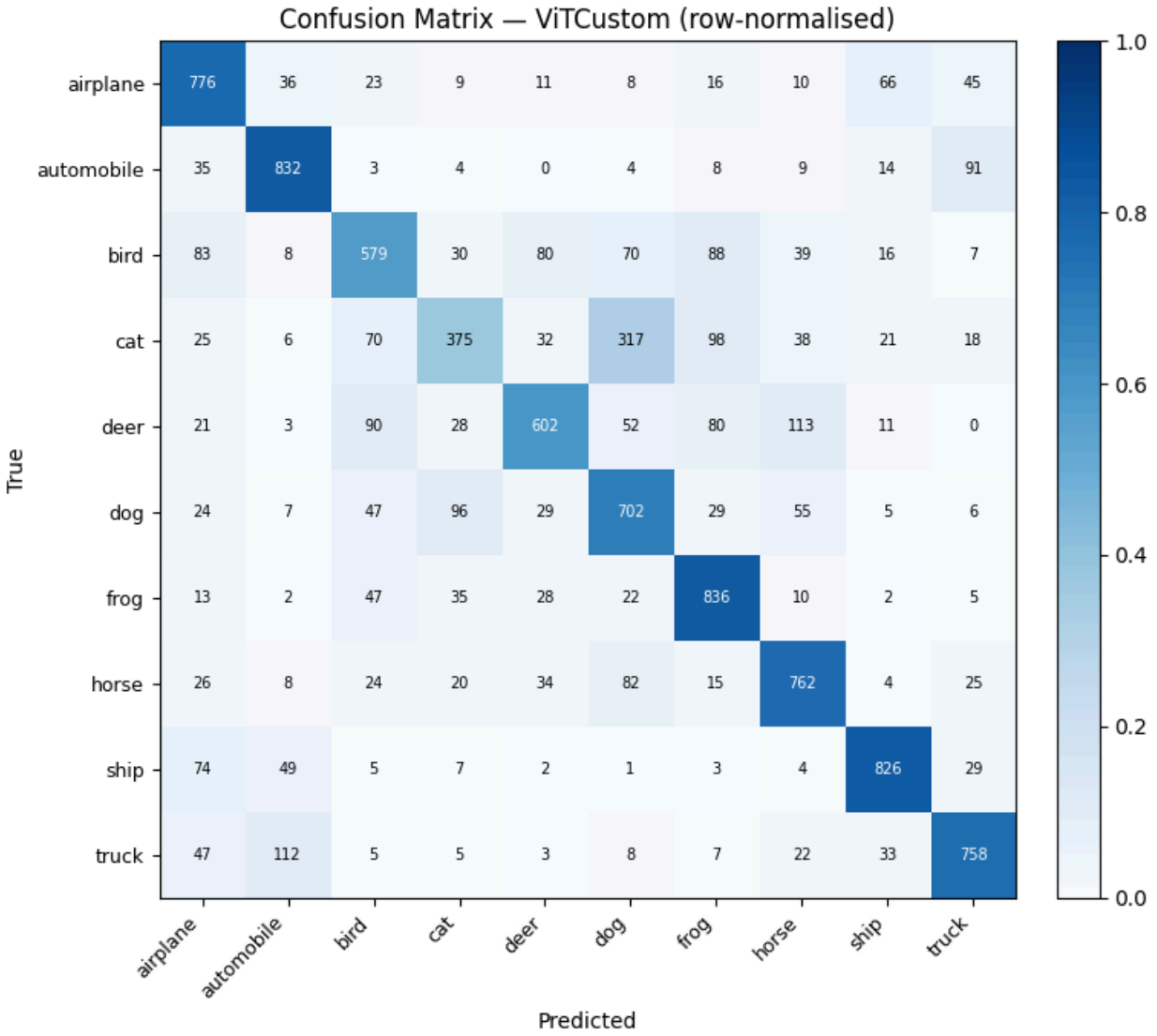
```
        bird     0.6628    0.5680    0.6117      1000
         cat     0.5991    0.3840    0.4680      1000
        deer     0.7126    0.6000    0.6515      1000
         dog     0.5526    0.7090    0.6211      1000
        frog     0.7540    0.8000    0.7763      1000
       horse     0.7004    0.7760    0.7362      1000
        ship     0.7939    0.8360    0.8144      1000
       truck     0.7710    0.7440    0.7573      1000

    accuracy                         0.7011     10000
   macro avg     0.7003    0.7011    0.6959     10000
weighted avg     0.7003    0.7011    0.6959     10000
```

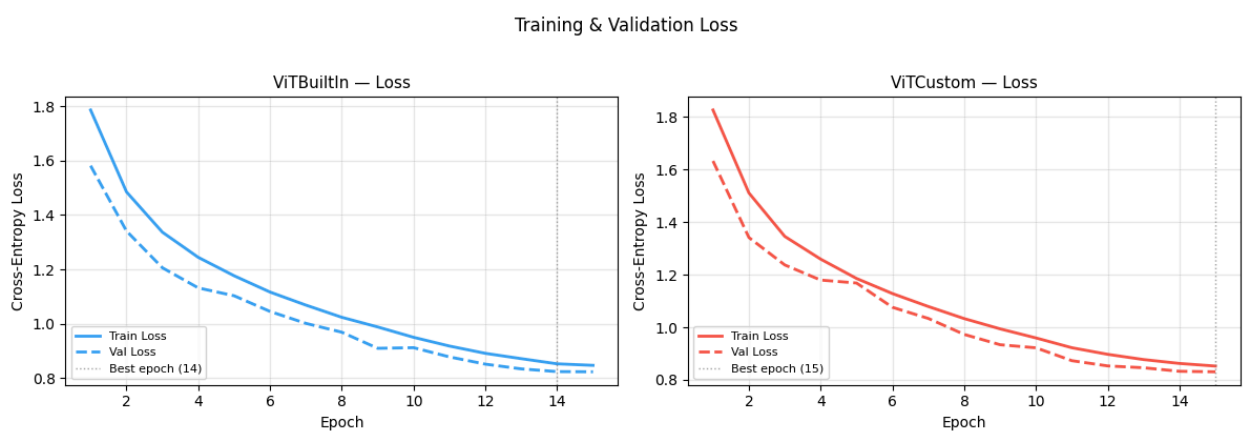
Classification Report — ViTCustom

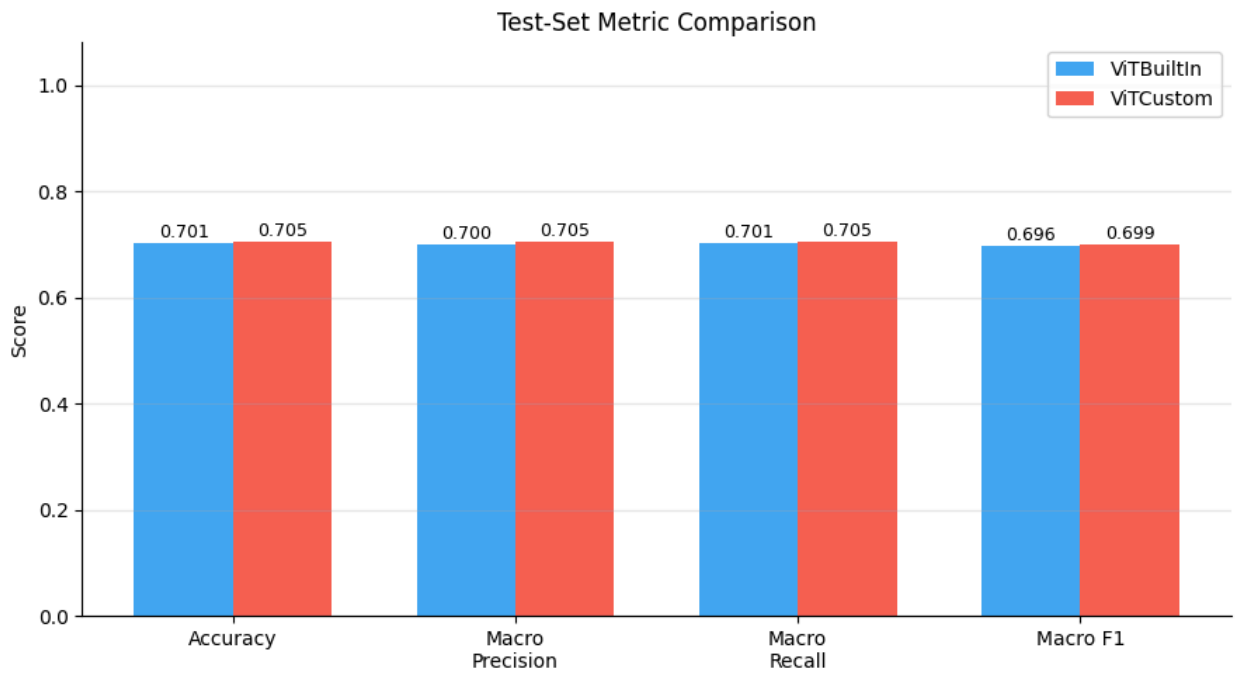
```
==============================================================
              precision    recall  f1-score   support

    airplane     0.6904    0.7760    0.7307      1000
  automobile     0.7827    0.8320    0.8066      1000
        bird     0.6484    0.5790    0.6117      1000
         cat     0.6158    0.3750    0.4661      1000
        deer     0.7333    0.6020    0.6612      1000
         dog     0.5545    0.7020    0.6196      1000
        frog     0.7085    0.8360    0.7670      1000
       horse     0.7175    0.7620    0.7391      1000
        ship     0.8277    0.8260    0.8268      1000
       truck     0.7703    0.7580    0.7641      1000
```

```
    accuracy                           0.7048     10000
   macro avg     0.7049    0.7048    0.6993     10000
weighted avg     0.7049    0.7048    0.6993     10000
```



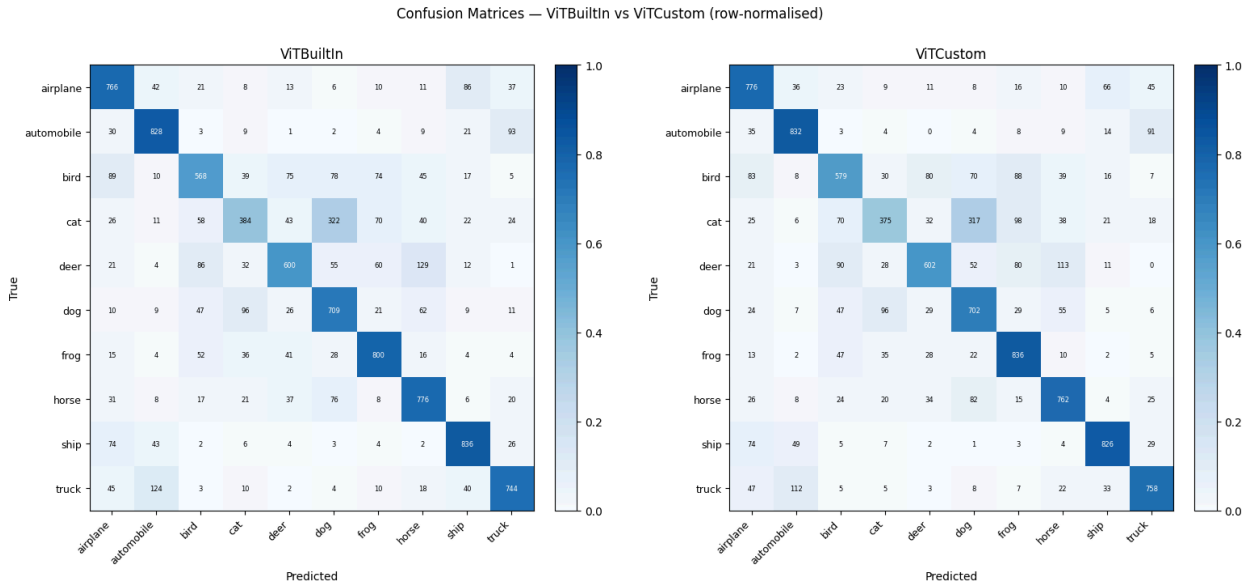

Confusion Matrix — ViTCustom (row-normalised)
True
Predicted
airplane 776 36 23 9 11 8 16 10 66 45
automobile 35 832 3 4 0 4 8 9 14 91
bird 83 8 579 30 80 70 88 39 16 7
cat 25 6 70 375 32 317 98 38 21 18
deer 21 3 90 28 602 52 80 113 11 0
dog 24 7 47 96 29 702 29 55 5 6
frog 13 2 47 35 28 22 836 10 2 5
horse 26 8 24 20 34 82 15 762 4 25
ship 74 49 5 7 2 1 3 4 826 29
truck 47 112 5 5 3 8 7 22 33 758
airplane automobile bird cat deer dog frog horse ship truck
1.0 0.8 0.6 0.4 0.2 0.0


Training & Validation Loss
ViTBuiltIn — Loss
Cross-Entropy Loss
Epoch
Train Loss
Val Loss
Best epoch (14)
ViTCustom — Loss
Cross-Entropy Loss
Epoch
Train Loss
Val Loss
Best epoch (15)



Additional observations:

- Built-in model benefits from optimized backend implementation
- Custom model may suffer from minor inefficiencies in attention computation
- Both models show similar learning trends, confirming correctness of implementation



5.5 Interpretation

The comparison between built-in and custom Vision Transformer implementations highlights several important insights.

First, **patch embedding and positional encoding are critical** for enabling Transformers to process image data. Without positional encoding, the model would lose spatial relationships between patches.

Second, the **built-in TransformerEncoder provides strong performance with minimal effort**, making it suitable for production systems and rapid prototyping. Its optimized implementation leads to better numerical stability and faster training.

In contrast, the **custom TransformerEncoder offers educational and research value**. By implementing attention mechanisms manually, we gain a deeper understanding of how Transformers operate internally, including:

- Query-Key-Value interactions
- Scaling factors in attention
- Role of residual connections and normalization

The slight performance gap between the two models can be attributed to:

- Optimization differences in PyTorch's internal implementation
- Sensitivity of custom models to initialization and normalization
- Potential inefficiencies in tensor operations

Overall, both models achieve comparable performance, demonstrating that the custom implementation successfully captures the essential behavior of Vision Transformers. However,

the built-in version remains preferable for practical applications, while the custom version is more valuable for learning and experimentation.

6. Hybrid architectures and sequence models

This section presents both hybrid CNN–Transformer models and GRU-based sequence models for image classification on CIFAR-10. The goal is to explore different ways of combining local feature extraction, global modeling, and sequential representations.

6.1 Hybrid architecture variants

Hybrid CNN–Transformer models combine the strengths of convolutional neural networks and Transformers. The CNN backbone is used to extract local spatial features, while the Transformer encoder models global relationships between tokens.

Two variants were implemented.

- The CNN+Transformer (64 tokens) model uses the 8×8 feature map directly, resulting in 64 tokens.
- The CNN+Transformer (16 tokens) model applies an additional pooling layer to reduce the feature map to 4×4, resulting in 16 tokens.

This design allows us to study the impact of tokenization size on performance.

6.2 Image-to-sequence design and GRU experiments

For sequence-based models, images must be converted into sequences. Two approaches were explored.

- The first approach is row-wise representation, where each row of the image is treated as a time step.
- The second approach is patch-wise representation, where the image is divided into small patches and each patch becomes a token.

These two methods provide different ways of representing spatial information in sequential form.

A GRU-based classifier was implemented for both sequence representations. The GRU processes the sequence and outputs a hidden representation, which is used for classification.

All models were trained for 10 epochs using the same experimental settings, ensuring fair comparison with other models in the project.

6.3 Results and comparisons

Hybrid CNN–Transformer models

Model	Train Acc	Test Acc	Train Loss	Test Loss	Time (s)
CNN+Transformer (64 tokens)	0.7745	0.7365	0.6370	0.7748	164.78
CNN+Transformer (16 tokens)	0.7895	0.7392	0.5965	0.7753	161.65

GRU-based models

Model	Train Acc	Test Acc	Train Loss	Test Loss	Time (s)
GRU (row-wise)	0.6509	0.5933	0.9737	1.1547	144.41
GRU (patch-wise)	0.6077	0.5759	1.0891	1.1714	143.25

The results show that hybrid models achieve significantly higher accuracy than GRU models.

The CNN+Transformer (64 tokens) model achieves slightly better test accuracy, while the CNN+Transformer (16 tokens) model achieves higher training accuracy.

For GRU models, the row-wise representation consistently outperforms the patch-wise representation.

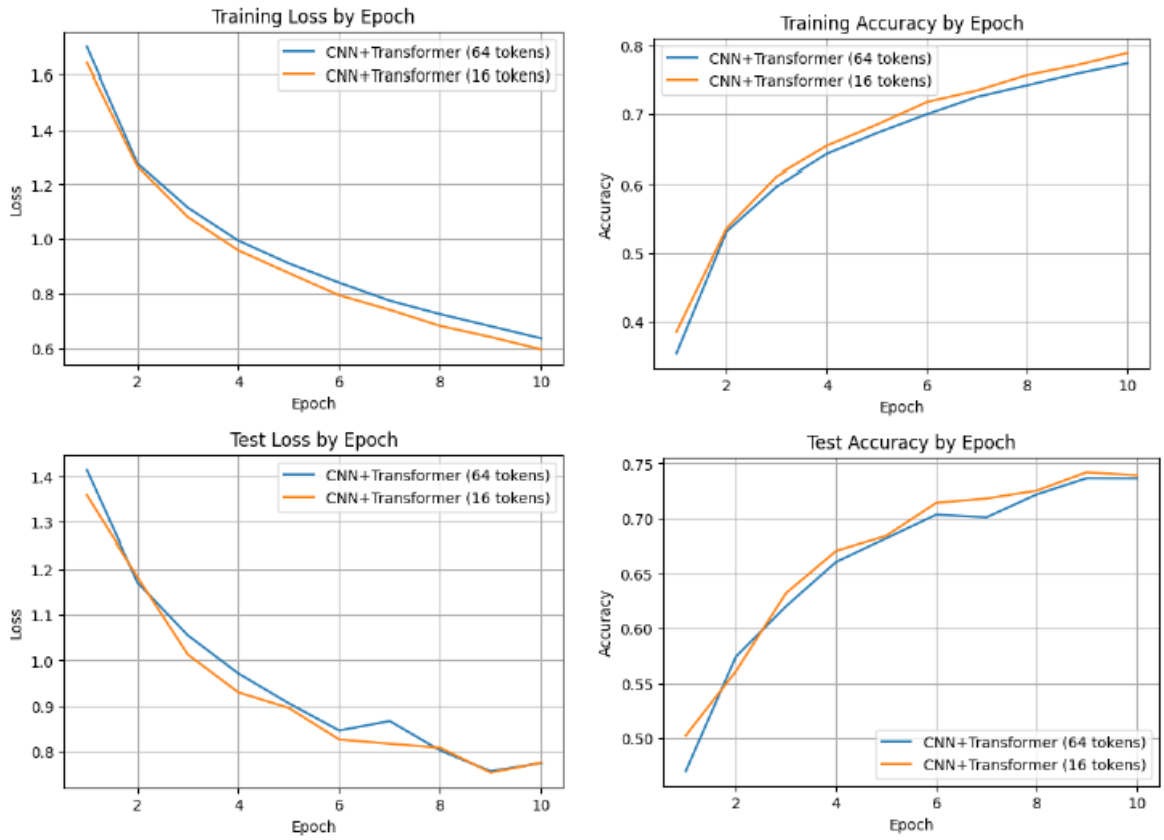


Figure 6.1 Training and test curves for CNN+Transformer (64 tokens) and (16 tokens).

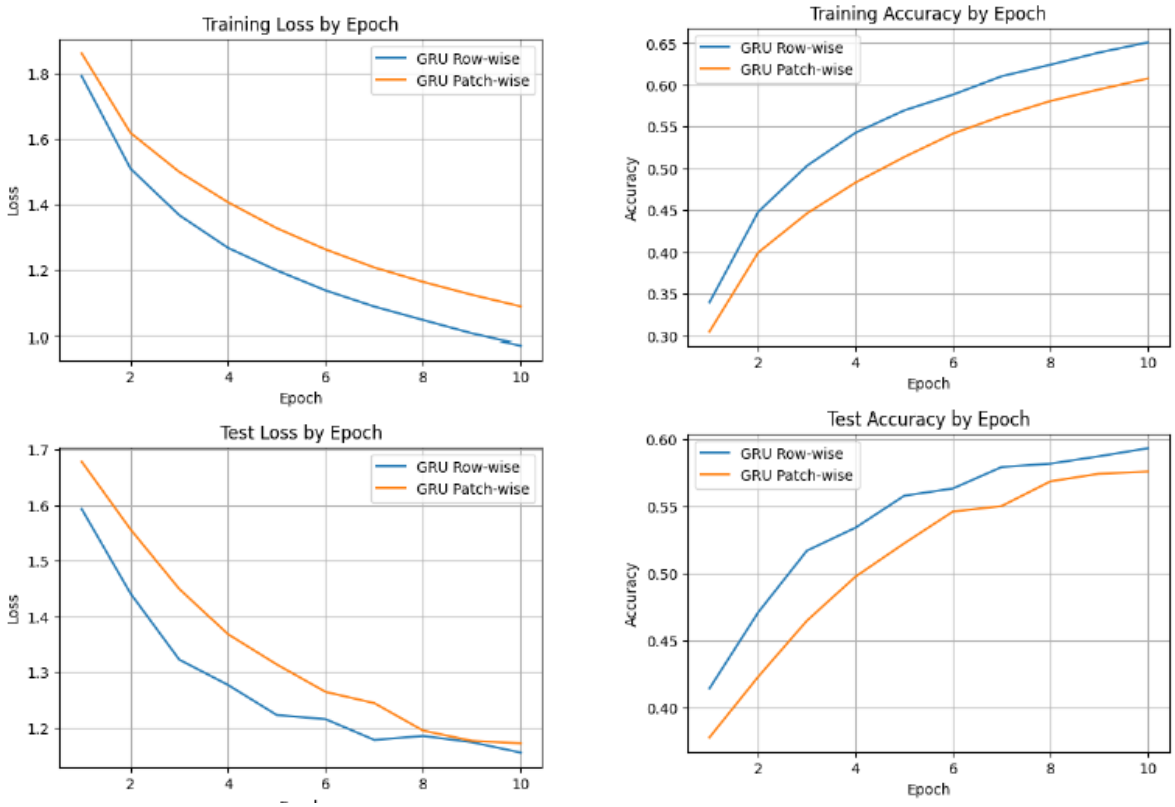


Figure 6.2 Training and test curves for GRU (row-wise) and GRU (patch-wise).

6.4 Interpretation

The hybrid CNN–Transformer models perform well because they combine local feature extraction and global modeling. The CNN+Transformer (64 tokens) model retains more spatial information and achieves slightly better generalization, while the CNN+Transformer (16 tokens) model is more compact and fits the training data better.

For sequence-based models, the row-wise representation is more effective than the patch-wise approach. However, both GRU models perform worse than CNN and Transformer-based models, showing the limitation of treating images purely as sequences without explicit spatial modeling.

7. Cross-model comparison

After completing all experiments, the final metrics from each model are summarized in Table 2. The table is designed to be simple, allowing results from independently developed pipelines to be combined without additional modifications.

Table 2. Final comparison across all group models

Model	Train Accuracy	Test Accuracy	Train Loss	Test Loss	Training Time (s)
Softmax Regression	0.4063	0.3622	1.7553	1.8698	171.26
MLP	0.7120	0.5366	0.8092	1.5452	265.49
CNN	**0.8955**	**0.7651**	0.2930	0.6980	141.06
ViT built-in	0.6974	0.7011	0.8471	0.8492	576
ViT custom	0.6951	0.7048	0.8525	0.8387	570
CNN + Transformer	**0.7745**	**0.7365**	0.6370	0.7748	164.78
GRU	0.6509	0.5933	0.9737	1.1547	144.41

8. Overall discussion

The results show a clear improvement from simple to advanced models.

Softmax Regression performs poorly, while MLP improves accuracy but still ignores spatial structure.

The CNN model achieves the best performance (76.51% test accuracy), confirming the importance of spatial feature extraction.

Vision Transformer models achieve competitive results (~70%), but require more complex design.

The hybrid CNN–Transformer models further improve performance (~73–74%), showing the benefit of combining local and global modeling.

In contrast, GRU-based models perform worse (~59%), indicating that treating images as sequences is less effective.

Overall, models that better capture spatial structure and combine local and global features achieve higher accuracy.

9. Conclusion

This assignment explored multiple deep learning models for image classification on the CIFAR-10 dataset, ranging from simple baseline models to more advanced architectures.

The results show that model performance improves significantly when spatial information is properly exploited. CNN and hybrid CNN–Transformer models achieve the best performance, while Vision Transformers provide competitive results. In contrast, sequence-based GRU models are less effective for image data.

Overall, the study highlights the importance of combining local feature extraction and global modeling. The hybrid approach offers a good balance between accuracy and model complexity, making it a strong solution for image classification tasks.